



NOVIG LABORATORIES
DESIGNATED CONTRACT MARKET
CONFIDENTIAL

DOCUMENT ID	COMMIT
THM-002	b9c2e26f18
STATUS	
Active	

Customer Support Agent

AI Automation Take-Home Assessment

tl;dr

Build a support ticket triage system that classifies incoming tickets, decides whether to auto-draft a response, and escalates sensitive cases to a human. Specifically, your system will:

- Classify each ticket by category and urgency level,
- Decide whether the LLM should draft a response or defer to a human,
- Generate a draft response when appropriate, and
- Include an eval harness that measures performance on a held-out set.

You have **72 hours** from receipt to submit a public URL to your implementation.

1 Overview

The objective is to showcase your ability to design, build, and evaluate an LLM-powered system that operates safely in a regulated, customer-facing context. We recommend spending **4–6 hours** on the exercise.

Novig is a CFTC-regulated peer-to-peer prediction market exchange. Our support team handles a wide mix of inbound tickets: routine product questions, account and money issues, and occasionally serious incidents like suspected account compromise or self-exclusion requests. The wrong automated response to a serious ticket is significantly worse than no response at all. This exercise is designed to test whether you can build a system that captures the obvious productivity wins of automation while knowing when to keep the LLM out of the loop.

2 Prompt

Build a system that, given an incoming support ticket, produces a structured output containing:

1. A **category** drawn from the taxonomy we provide,
2. An **urgency level**: low, medium, high, or `escalate_immediately`,
3. A **drafted response**, OR an explicit decision not to draft (with a reason), and
4. A **confidence signal** of your choosing.

We will provide:

- `tickets_train.jsonl` — 30 labeled tickets with category, urgency, whether a draft is appropriate, and notes on the gold response,
- `tickets_eval.jsonl` — 15 held-out tickets with labels hidden, and
- `taxonomy.md` — the category list, urgency level descriptions, and the output schema your system should emit.

Your repo should include a single command (e.g., `make eval` or `python eval.py`) that runs your system over `tickets_eval.jsonl` and writes predictions to `predictions.jsonl` in the schema defined in `taxonomy.md`. How you structure the eval loop itself is part of what we are evaluating.

3 Constraints

3.1 User Need

The system serves the Novig support team. They need help drafting responses to high-volume, low-risk tickets so they can spend more time on the cases that actually require human judgment.

The cost of a bad auto-draft on a sensitive ticket (e.g., a user reporting account compromise, mentioning self-harm, threatening legal action, or signaling problem gambling) is much higher than the cost of failing to auto-draft an easy one. Your system should reflect that asymmetry.

3.2 Evaluation Harness

You must ship an eval harness alongside your system. At minimum it should report:

- Category classification accuracy,
- Urgency classification accuracy,
- A confusion matrix for urgency, and
- A metric capturing whether the system drafted a response when it should have declined (false-draft rate on sensitive tickets is the most important number in this exercise).

We expect the evals to be more than a notebook afterthought. Build them early; iterate against them.

3.3 Safety Behavior

The taxonomy defines the conditions under which your system must not draft a response, plus a broader set of conditions where deferring to a human is the right call (factual disputes, definitive policy statements, jurisdictional eligibility). The hard rules are non-negotiable; the soft rules require judgment. Errors on the hard rules will heavily penalize your submission. The training set contains examples of both.

3.4 Technology

Use any LLM provider and any language (Python or TypeScript recommended). No fine-tuning — prompting, few-shot, routing, and multi-step approaches are all fair game. You do not need a vector database for a dataset this size; if you reach for one, justify it in the writeup. We will reimburse up to \$20 in API costs against a screenshot.

3.5 Scope Discipline

Do not build a UI. Do not build a sprawling agent. We would much rather see a clean, well-evaluated system with a clear interface than a feature-rich one with no evidence that it works. If you find yourself past six hours, ship what you have and document what you would do next.

3.6 Writeup

Include a `WRITEUP.md` of at most one page covering:

- Your approach and the key design decisions,
- What your evals showed (numbers, not vibes),
- Where the system fails and why, and
- What you would build next given another week.

3.7 Time

You have **72 hours** from receiving this exercise to submit a URL to the public GitHub repository where your project lives. We recommend 4–6 hours of actual work within that window.

4 Evaluation

The expectation is that the system works and captures the requirements above. We are looking for more than a bare-bones MVP, but we are explicitly not looking for over-engineering. Your submission will be evaluated against the following criteria, roughly in order of weight:

1. **Eval rigor:** Did you build evals that actually measure what matters? Are they reproducible? Did you iterate against them?
2. **Sensitive-case handling:** Does the system correctly decline to auto-draft on the tickets where a human must respond? This is the single most important behavior.
3. **Code quality:** Is the system something a teammate could pick up, read, and modify? Is the predict interface clean? Is error handling sensible?
4. **Classification accuracy:** Category and urgency accuracy on the held-out set.
5. **Writeup:** Can you reason clearly about tradeoffs, failure modes, and what you would do next?

A submission with 75% accuracy that correctly routes every sensitive ticket and ships thoughtful evals will beat a submission with 90% accuracy that cheerfully drafts a response to a hacking report.

5 Deliverable

When you are done, upload the following into the Ashby submission link:

- A link to a public GitHub repository containing your source code, eval harness, eval results (predictions.jsonl plus reported metrics), and WRITEUP.md,
- A screenshot of your API usage / cost dashboard for reimbursement (optional).